10

多处理器调度（进阶）

本章介绍**多处理器调度**（multiprocessor scheduling）的基础知识。这个话题比较深，最好在你较为深入地学习了并发（concurrency，也就是本书的第二个“简单部分”）之后再来读。

多处理器（multiprocessor）系统多年来只存在于计算领域的高端，如今却越来越常见，已经进入台式机、笔记本电脑，甚至移动设备。这种普及源于**多核**（multicore）处理器的兴起：多个 CPU 核心被封装在同一块芯片上。这类芯片之所以流行，是因为计算机架构师发现，要让单个 CPU 快很多，又不消耗（远远）过多的电能，实在太难。于是，现在我们每个人手上都有好几个 CPU 可用，这是件好事，对吧？

当然，CPU 不止一个之后，也带来了许多难题。其中最主要的一个是：典型的应用程序（比如你写的某个 C 程序）只会用到一个 CPU；加再多 CPU，这个应用程序也不会跑得更快。要解决这个问题，你得重写应用程序，让它能**并行**（parallel）运行，比如使用**线程**（threads），这在本书第二部分会详细讨论。多线程（multi-threaded）应用程序可以把工作分散到多个 CPU 上，因此给它更多 CPU 资源，它就能跑得更快。

除了应用程序，操作系统也遇到了一个新问题（这并不奇怪！），那就是**多处理器调度**（multiprocessor scheduling）。到目前为止，我们已经讨论了单处理器调度背后的许多原则；怎样把这些思想推广到多个 CPU 上？我们又必须克服哪些新问题？于是，我们的问题是：

关键问题：如何在多个 CPU 上调度作业

操作系统应该如何在多个 CPU 上调度作业？会出现哪些新问题？原来那些老技术还管用吗，还是需要新的思路？

补充：进阶章节

要真正读懂进阶章节，需要用到本书很大一部分内容；但从逻辑上讲，这些章节又属于比这些前置内容更靠前的部分。例如，本章讲多处理器调度（multiprocessor scheduling），如果你先读过中间关于并发的部分，理解起来会容易得多；但从逻辑上讲，它属于本书讲虚拟化（virtualization）的部分，具体来说是讲 CPU 调度的那部分。因此，建议不按顺序阅读这类章节；就本章而言，建议在读完本书第二部分之后再读。

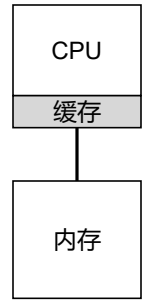


图 10.1：**带缓存的单个 CPU**

10.1 背景：多处理器架构

要理解多处理器调度（multiprocessor scheduling）面临的新问题，就得先理解单 CPU 硬件和多 CPU 硬件之间一个全新的根本区别。这个区别的核心在于硬件**缓存**（cache）的使用方式（如图 10.1），以及数据究竟如何在多个处理器之间共享。下面我们从较高的层次进一步讨论这个问题。更多细节可以在别处找到 [CSG99]，尤其是高年级本科或研究生的计算机体系结构课程。

在只有一个 CPU 的系统中，有一个由**硬件缓存**（hardware cache）组成的层次结构，一般来说，它们能帮助处理器更快地运行程序。缓存是一些小而快的存储器，（一般来说）保存着系统主存中常用数据的副本。相比之下，主存保存着全部数据，但访问这块更大的存储器要慢一些。通过把经常访问的数据放在缓存里，系统就能让又大又慢的存储器看起来像是快的。

举个例子：假设一个程序发出一条显式的加载指令，从内存中取一个值，而系统很简单，只有一个 CPU；这个 CPU 有一个小缓存和一个大的主存。程序第一次发出这条加载指令时，数据在主存里，因此要花很长时间才能取到（可能要几十纳秒，甚至几百纳秒）。处理器预计这个数据可能会被再次使用，于是把取来的数据复制一份放进 CPU 缓存。如果程序之后再次读取同一个数据，CPU 会先在缓存里找；如果找到了，取数据就快得多（比如只要几纳秒），程序也就跑得更快。

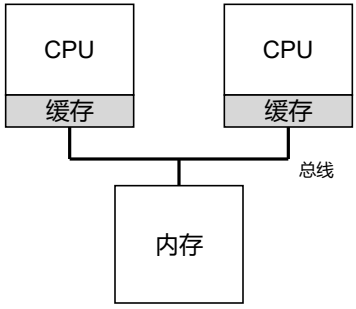


图 10.2：**两个带缓存的 CPU 共享内存**

 © 2008–25, ARPACI-DUSSEAU

因此，缓存建立在**局部性**（locality）这个概念之上。局部性有两种：**时间局部性**（temporal locality）和**空间局部性**（spatial locality）。时间局部性的思想是：一个数据被访问过之后，很可能在不久的将来再次被访问；想象一下循环里被一遍又一遍访问的变量，甚至指令本身。空间局部性的思想是：如果程序访问了地址 x 处的数据，那么它很可能也会访问 x 附近的数据；想想程序顺序遍历一个数组，或者指令一条接一条地执行。由于许多程序都具有这几种局部性，硬件系统就能比较准确地猜出该把哪些数据放进缓存，因而效果很好。

接下来是棘手的部分：如果一个系统里有多个处理器，共享同一个主存（如图 10.2 所示），会怎么样？

事实证明，有多个 CPU 时，缓存要复杂得多。例如，假设运行在 CPU 1 上的程序读取了地址 A 处的一个数据（值为 D）；由于 CPU 1 的缓存里没有这个数据，系统就从主存中取出它，得到值 D。然后程序修改了地址 A 处的值，只是把新值 D' 写进了自己的缓存；把数据一路写到主存很慢，所以系统（通常）会晚些再做。接着，假设操作系统决定停止运行这个程序，把它挪到 CPU 2 上。程序再次读取地址 A 处的值；CPU 2 的缓存里没有这个数据，于是系统从主存中取，得到的是旧值 D，而不是正确的值 D'。糟糕！

这个一般性问题叫作**缓存一致性**（cache coherence）问题，关于如何解决它的种种微妙之处，有大量的研究文献 [SHW11]。这里我们略过所有细节，只讲几个要点；想了解更多，就去上一门（或者三门）计算机体系结构课吧。

基本的解决方案由硬件提供：通过监视内存访问，硬件基本上可以确保“正确的事情”发生，并维持只有一个共享内存（shared memory）的视图。在基于总线（bus）的系统中（如上所述），实现这一点的一种方法是使用一种叫作**总线窥探**（bus snooping）的老技术 [G83]：每个缓存都观察把它们连到主存的那条总线，以此关注内存的更新。当某个 CPU 看到自己缓存中某个数据被更新时，它就会注意到这个变化，要么让自己的副本**失效**（invalidate，即把它从自己的缓存中删除），要么**更新**（update）它（即把新值也放进自己的缓存）。前面暗示过，回写缓存（write-back cache）会让这件事更复杂（因为写到主存的操作要过一阵才能看见），但你可以想象这个基本方案是如何工作的。

10.2 别忘了同步

既然缓存已经做了这么多工作来提供一致性，那么程序（或操作系统本身）访问共享数据时，还需要操心什么吗？遗憾的是，答案是肯定的，本书讲并发的第二部分对此有非常详细的说明。这里我们不深入细节，只简单勾勒（或回顾）一下其中的一些基本思想（假定你已经熟悉并发）。

在多个 CPU 之间访问（尤其是更新）共享的数据项或数据结构（data structure）时，很可能需要使用锁（lock）这类互斥（mutual exclusion）原语来保证正确性（其他方法，比如构建**无锁**（lock-free）数据结构，很复杂，只在少数情况下使用；详见并发部分讲死锁（deadlock）的那一章）。例如，假设有一个共享队列（queue），正在被多个 CPU 并发访问。如果没有锁，即使有底层的一致性协议（coherence protocol），并发地（concurrently）往队列里添加或删除元素也不会按预期工作；需要用锁来原子地（atomically）把数据结构更新到新状态。

 © 2008–25, ARPACI-DUSSEAU

```
1  typedef struct __Node_t {
2      int              value;
3      struct __Node_t *next;
4  } Node_t;
5
6  int List_Pop() {
7      Node_t *tmp = head;           // 记住原来的头节点
8      int value   = head->value;    // .....以及它的值
9      head        = head->next;     // 前进到下一个节点
10     free(tmp);                    // 释放原来的头节点
11     return value;                 // 返回头节点的值
12 }
```

图 10.3：**简单的链表删除代码**

具体来说，设想有这样一段代码，用来从一个共享链表（linked list）中删除一个元素，如图 10.3 所示。假设两个 CPU 上的线程同时进入这个例程。如果线程 1 执行了第一行，它的 `tmp` 变量里就保存了 `head` 的当前值；如果线程 2 接着也执行了第一行，它自己私有的 `tmp` 变量里也会保存同一个 `head` 值（`tmp` 是在栈上分配的，因此每个线程都有自己私有的一份）。于是，两个线程不是各自从链表头部删除一个元素，而是都试图删除同一个头部元素，从而导致各种问题（比如在第 10 行试图把头部元素释放两次，还可能把同一个数据值返回两次）。

当然，解决办法是通过**加锁**（locking）让这类例程变得正确。在这个例子中，分配一个简单的互斥锁（mutex，例如 `pthread_mutex_t m;`），然后在例程开头加上 `lock(&m)`、在末尾加上 `unlock(&m)`，就能解决问题，确保代码按预期执行。遗憾的是，我们会看到，这种做法并非没有问题，尤其是在性能方面。具体来说，随着 CPU 数量增加，访问一个同步的共享数据结构会变得相当慢。

10.3 最后一个问题：缓存亲和性

构建多处理器调度程序（multiprocessor scheduler）时，还会出现最后一个与缓存有关的问题，叫作**缓存亲和性**（cache affinity）[TTG95]。这个概念很简单：一个进程（process）在某个 CPU 上运行时，会在这个 CPU 的缓存和 TLB 中积累相当多的状态。下次这个进程运行时，让它在同一个 CPU 上运行往往是有利的，因为如果它的部分状态已经在那个 CPU 的缓存里了，它就会跑得更快。反之，如果每次都让进程在不同的 CPU 上运行，进程的性能就会变差，因为它每次运行都得重新加载状态（注意，多亏了硬件的缓存一致性协议（cache coherence protocol），它在不同的 CPU 上也能正确运行）。因此，多处理器调度程序（multiprocessor scheduler）在做调度决策时应该考虑缓存亲和性（cache affinity），只要有可能，就尽量让进程留在同一个 CPU 上。

10.4 单队列调度

有了这些背景知识，下面我们讨论如何为多处理器系统构建调度程序（scheduler）。最基本的方法是直接沿用单处理器调度的基本框架，把所有需要调度的作业放进一个队列；我们把这种方法叫作**单队列多处理器调度**（single-queue multiprocessor scheduling），简称**SQMS**。这种方法的优点是简单：拿一个现成的、能挑出下一个最该运行的作业的策略，改造成能在多个 CPU 上工作的版本（比如有两个 CPU 时，就挑出两个最该运行的作业），并不需要多少工作量。

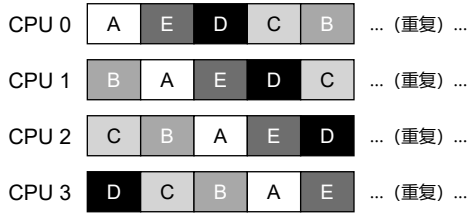
然而，SQMS 有明显的缺点。第一个问题是缺乏**可扩展性**（scalability）。为了确保调度程序在多个 CPU 上正确工作，开发者会像前面说的那样，在代码里加入某种形式的**锁**（locking）。锁能确保 SQMS 的代码访问那个唯一的队列时（比如找下一个要运行的作业），得到正确的结果。

遗憾的是，锁会大大降低性能，尤其是在系统中 CPU 数量增多时 [A90]。随着对这把唯一的锁的争用（contention）加剧，系统花在锁开销上的时间越来越多，花在本该做的工作上的时间越来越少（注：要是哪天能在这里放上真实的测量数据就好了）。

SQMS 的第二个主要问题是缓存亲和性（cache affinity）。例如，假设有五个作业要运行（A、B、C、D、E），有四个处理器。那么调度队列（scheduling queue）是这样的：

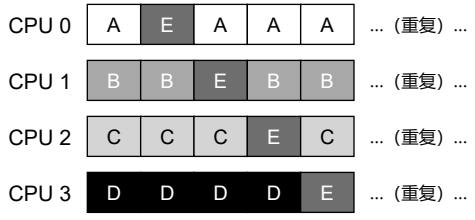


随着时间推移，假设每个作业运行一个时间片（time slice），然后换另一个作业，下面是一种可能的各 CPU 作业调度：



由于每个 CPU 只是从全局共享的队列里挑下一个要运行的作业，每个作业最终都在各个 CPU 之间来回跳，这恰恰和缓存亲和性（cache affinity）的要求背道而驰。

为了解决这个问题，大多数 SQMS 调度程序都包含某种亲和性（affinity）机制，尽量让进程更有可能继续在同一个 CPU 上运行。具体来说，可以为某些作业保持亲和性（affinity），而挪动另一些作业来平衡负载。例如，设想同样这五个作业按下面这样调度：



 © 2008–25, ARPACI-DUSSEAU

在这种安排下，作业 *A* 到 *D* 都不在处理器之间移动，只有作业 *E* 在 CPU 之间**迁移**（migrating），从而为大多数作业保持了亲和性（affinity）。下一轮你可以决定迁移另一个作业，这样也能实现某种亲和性（affinity）上的公平。不过，实现这样的方案可能会很复杂。

由此可见，SQMS 方法有优点也有缺点。如果已经有一个单 CPU 调度程序，按定义它只有一个队列，那么实现 SQMS 很直接。但是，它的扩展性不好（因为有同步开销），也不太容易保持缓存亲和性（cache affinity）。

10.5 多队列调度

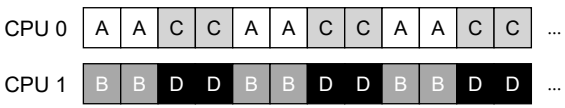
由于单队列调度程序（single-queue scheduler）存在这些问题，有些系统选择使用多个队列，比如每个 CPU 一个。我们把这种方法叫作**多队列多处理器调度**（multi-queue multiprocessor scheduling，**MQMS**）。

在 MQMS 中，基本的调度框架由多个调度队列组成。每个队列可能遵循某种特定的调度规则（scheduling discipline），比如时间片轮转（Round-Robin），当然也可以使用任何算法。作业进入系统时，会根据某种启发式规则（比如随机挑选，或者挑一个作业比其他队列少的队列）被放进恰好一个调度队列。之后，它基本上是独立调度的，从而避免了单队列（single-queue）方法中信息共享和同步（synchronization）带来的问题。

例如，假设系统只有两个 CPU（标记为 CPU 0 和 CPU 1），有若干作业进入系统：比如 *A*、*B*、*C* 和 *D*。由于现在每个 CPU 都有一个调度队列，操作系统必须决定把每个作业放进哪个队列。它可能会这样放：

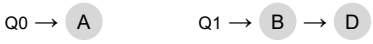


根据队列的调度策略（scheduling policy），现在每个 CPU 在决定运行什么时，都有两个作业可选。例如，采用**时间片轮转**（round robin）时，系统可能产生这样的调度：

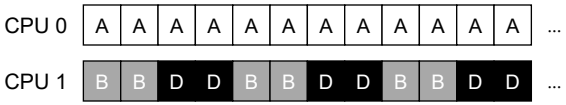


MQMS 相对于 SQMS 有一个明显的优势：它天生就应该更具可扩展性。随着 CPU 数量增加，队列的数量也随之增加，因此锁争用（lock contention）和缓存争用（cache contention）不应成为核心问题。此外，MQMS 本身就提供了缓存亲和性（cache affinity）：作业留在同一个 CPU 上，因而能享受重用其中缓存内容的好处。

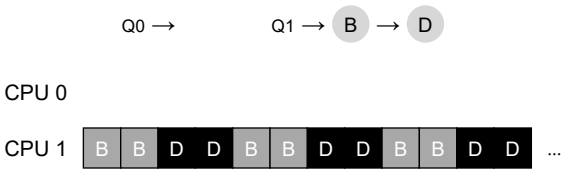
不过，如果你一直留心的话，可能已经发现我们有了一个新问题，而且这是基于多队列（multi-queue）的方法所固有的问题：**负载不均**（load imbalance）。假设设置和上面一样（四个作业、两个 CPU），但随后其中一个作业（比如 *C*）结束了。现在调度队列变成了这样：



如果接着在系统的每个队列上运行时间片轮转（Round-Robin）策略，就会得到这样的调度：



从图中可以看到，*A* 得到的 CPU 时间是 *B* 和 *D* 的两倍，这不是我们想要的结果。更糟的是，假设 *A* 和 *C* 都结束了，系统中只剩下作业 *B* 和 *D*。两个调度队列以及由此得到的时间线会是这样：



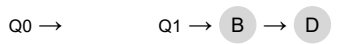
太糟糕了，CPU 0 闲着！（此处插入悲壮而阴森的音乐）于是我们的 CPU 使用时间线看起来相当凄惨。

那么，可怜的多队列多处理器调度程序（multi-queue multiprocessor scheduler）该怎么办？我们怎样才能克服负载不均（load imbalance）这个隐蔽的问题，打败……霸天虎[1]的邪恶势力？我们怎样才能不再问这些和这本原本很精彩的书几乎毫不相干的问题？

关键问题：如何应对负载不均
多队列多处理器调度程序（multi-queue multiprocessor scheduler）应该如何应对负载不均（load imbalance），以便更好地实现它的调度目标？

这个问题显而易见的答案是把作业挪来挪去，这种技术我们（又一次）称为**迁移**（migration）。通过把作业从一个 CPU 迁移到另一个 CPU，就能实现真正的负载均衡（load balance）。

我们来看几个例子，把问题讲清楚一些。同样，一个 CPU 闲着，另一个上面有一些作业。



在这种情况下，该怎么迁移很好理解：操作系统只要把 *B* 或 *D* 中的一个挪到 CPU 0 上就行了。这一次作业迁移的结果是负载均衡（load balance），皆大欢喜。

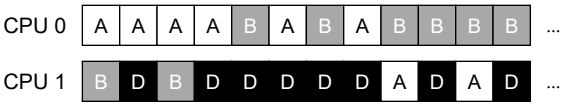
更棘手的情况出现在前面那个例子里：*A* 独自留在 CPU 0 上，*B* 和 *D* 在 CPU 1 上交替运行。



在这种情况下，迁移一次解决不了问题。你会怎么做？可惜，答案是持续地迁移一个或多个作业。一种可能的办法是不断切换作业，如下面的时间线所示。图中，一开始 *A* 独自

[1]鲜为人知的事实：塞伯坦星球就是被糟糕的 CPU 调度决策毁掉的。就让这成为本书第一次也是最后一次提到变形金刚吧，我们为此真诚地道歉。

在 CPU 0 上，*B* 和 *D* 在 CPU 1 上交替运行。几个时间片之后，*B* 被挪到 CPU 0 上和 *A* 竞争，而 *D* 则在 CPU 1 上独享几个时间片。这样负载就均衡了：



当然，可能的迁移模式还有很多。但现在到了棘手的部分：系统该如何决定迁移？

一种基本的方法是**工作窃取**（work stealing）[FLR98]：作业较少的（源）队列时不时看一眼另一个（目标）队列有多满；如果目标队列（明显）比源队列满，源队列就从目标队列“窃取”一个或多个作业，以平衡负载。

当然，这种方法有一种天然的矛盾：查看其他队列太频繁，开销就高，也难以扩展，而可扩展恰恰是当初实现多队列（multi-queue）调度的全部目的！反过来，查看得太少，又可能出现严重的负载不均（load imbalance）。和系统策略设计中常见的情况一样，找到合适的阈值仍然是一门玄妙的手艺。

10.6 Linux 多处理器调度程序

有意思的是，在 Linux 社区里，构建多处理器调度程序（multiprocessor scheduler）一直没有形成统一的方案。随着时间推移，先后出现了三种不同的调度程序：O(1) 调度程序、完全公平调度程序（Completely Fair Scheduler，CFS）和 BF 调度程序（BFS）[2]。Meehean 的博士论文很好地概述了这几种调度程序的优缺点 [M11]；这里我们只总结几点基本情况。

O(1) 和 CFS 都使用多个队列，而 BFS 使用单个队列，可见两种方法都能成功。当然，它们在其他细节上还有许多差别。例如，O(1) 是基于优先级（priority）的调度程序（类似前面讲过的 MLFQ），它随时间调整进程的优先级，并调度优先级最高的进程，以实现各种调度目标，尤其注重交互性（interactivity）。CFS 则是一种确定性的比例份额（proportional-share）方法，更像前面讲过的步长调度（stride scheduling）。BFS 是三者中唯一的单队列（single-queue）方法，它也是比例份额（proportional-share）的，但基于一种更复杂的方案，叫作 Earliest Eligible Virtual Deadline First（EEVDF）[SA96]。这些现代算法请自己去读一读；现在你应该能看懂它们是怎么工作的了！

10.7 小结

本章介绍了多处理器调度（multiprocessor scheduling）的各种方法。单队列方法（SQMS）实现简单，负载均衡得好，但天生难以扩展到很多处理器，也难以保持缓存亲和性（cache affinity）。多队列方法（MQMS）扩展性更好，也能很好地处理缓存亲和性（cache affinity），但有负载不均（load imbalance）的问题，而且更复杂。无论哪种方法都没有简单的答案：构建通用的调度程序仍是一项艰巨的任务，因为代码上的小改动就可能导致行为上的巨大差异。除非你确切知道自己在做什么，或者至少有人为此付你一大笔钱，否则别去碰这件事。

[2] BF 代表什么，请自己去查；先提醒一句，胆小的人慎查。

参考文献

[A90] "The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors" by Thomas E. Anderson. IEEE TPDS Volume 1:1, January 1990. *A classic paper on how different locking alternatives do and don't scale. By Tom Anderson, very well known researcher in both systems and networking. And author of a very fine OS textbook, we must say.*

[B+10] "An Analysis of Linux Scalability to Many Cores Abstract" by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. *A terrific modern paper on the difficulties of scaling Linux to many cores.*

[CSG99] "Parallel Computer Architecture: A Hardware/Software Approach" by David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. Morgan Kaufmann, 1999. *A treasure filled with details about parallel machines and algorithms. As Mark Hill humorously observes on the jacket, the book contains more information than most research papers.*

[FLR98] "The Implementation of the Cilk-5 Multithreaded Language" by Matteo Frigo, Charles E. Leiserson, Keith Randall. PLDI '98, Montreal, Canada, June 1998. *Cilk is a lightweight language and runtime for writing parallel programs, and an excellent example of the work-stealing paradigm.*

[G83] "Using Cache Memory To Reduce Processor-Memory Traffic" by James R. Goodman. ISCA '83, Stockholm, Sweden, June 1983. *The pioneering paper on how to use bus snooping, i.e., paying attention to requests you see on the bus, to build a cache coherence protocol. Goodman's research over many years at Wisconsin is full of cleverness, this being but one example.*

[M11] "Towards Transparent CPU Scheduling" by Joseph T. Meehean. Doctoral Dissertation at University of Wisconsin—Madison, 2011. *A dissertation that covers a lot of the details of how modern Linux multiprocessor scheduling works. Pretty awesome! But, as co-advisors of Joe's, we may be a bit biased here.*

[SHW11] "A Primer on Memory Consistency and Cache Coherence" by Daniel J. Sorin, Mark D. Hill, and David A. Wood. Synthesis Lectures in Computer Architecture. Morgan and Claypool Publishers, May 2011. *A definitive overview of memory consistency and multiprocessor caching. Required reading for anyone who likes to know way too much about a given topic.*

[SA96] "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation" by Ion Stoica and Hussein Abdel-Wahab. Technical Report TR-95-22, Old Dominion University, 1996. *A tech report on this cool scheduling idea, from Ion Stoica, now a professor at U.C. Berkeley and world expert in networking, distributed systems, and many other things.*

[TTG95] "Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors" by Josep Torrellas, Andrew Tucker, Anoop Gupta. Journal of Parallel and Distributed Computing, Volume 24:2, February 1995. *This is not the first paper on the topic, but it has citations to earlier work, and is a more readable and practical paper than some of the earlier queuing-based analysis papers.*

作业（模拟）

在这个作业中，我们将用 `multi.py` 模拟一个多处理器 CPU 调度程序，并了解它的一些细节。关于这个模拟器及其选项的更多信息，请阅读相关的 README。

问题

1. 首先，我们来学习如何用这个模拟器研究怎样构建一个高效的多处理器调度程序（multiprocessor scheduler）。第一个模拟只运行一个作业，它的运行时间为 30，工作集（working set）大小为 200。在一个模拟的 CPU 上运行这个作业（这里叫作作业“a”），命令如下：

   ```
   ./multi.py -n 1 -L a:30:200
   ```

   它需要多长时间才能完成？打开 `-c` 选项可以看到最终答案，打开 `-t` 选项可以看到这个作业逐个时钟滴答的跟踪，以及它是如何被调度的。

2. 现在增大缓存，让这个作业的工作集（working set，大小为 200）能放进缓存（默认大小为 100）；例如，运行：

   ```
   ./multi.py -n 1 -L a:30:200 -M 300
   ```

   你能预测出作业放进缓存之后会运行多快吗？（提示：记住预热速率这个关键参数，它由 `-r` 选项设置。）打开求解选项（`-c`）运行，检查你的答案。

3. `multi.py` 有一个很酷的地方：你可以用不同的跟踪选项看到更多细节。运行和上面相同的模拟，但这次打开剩余时间跟踪（`-T`）。这个选项会显示每个时间步调度到 CPU 上的作业，以及每个时钟滴答之后这个作业还剩多少运行时间。你注意到第二列是怎样递减的吗？

4. 现在再加一项跟踪，用 `-C` 选项显示每个 CPU 缓存对每个作业的状态。对每个作业，每个缓存要么显示一个空格（缓存对这个作业是冷的），要么显示一个“w”（缓存对这个作业是热的）。在这个简单的例子里，缓存在什么时候对作业“a”变热？把预热时间参数（`-w`）改成比默认值小或大的值，会发生什么？

5. 到现在，你应该已经很清楚模拟器在单个 CPU 上运行单个作业时是怎么工作的了。不过，这不是讲多处理器 CPU 调度的一章吗？没错！那我们开始处理多个作业吧。具体来说，在一个双 CPU 系统上运行下面三个作业，即输入：

   ```
   ./multi.py -n 2 -L a:100:100,b:100:50,c:100:50
   ```

   在集中式的时间片轮转（Round-Robin）调度程序下，你能预测出这需要多长时间吗？用 `-c` 看看你对不对，然后用 `-t` 深入看看逐步的细节，再用 `-C` 看看这些作业的缓存是否被有效地预热了。你注意到了什么？

6. 现在我们施加一些显式的控制，来研究本章讲到的**缓存亲和性**（cache affinity）。为此，你需要用到 `-A` 选项。这个选项可以限制调度程序能把某个作业放在哪些 CPU 上。这

里，我们用它把作业“b”和“c”放在 CPU 1 上，而把“a”限制在 CPU 0 上。要实现这个魔法，输入：

```
./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 \
    -A a:0,b:1,c:1
```

别忘了打开各种跟踪选项，看看到底发生了什么！你能预测这个版本会运行多快吗？它为什么表现更好？把“a”“b”“c”以其他组合放到两个处理器上，会运行得更快还是更慢？

7. 带缓存的多处理器有一个有意思的地方：与在单个处理器上运行作业相比，使用多个 CPU（及其缓存）时，作业有机会获得比预期更好的加速。具体来说，在 *N* 个 CPU 上运行时，有时可以获得超过 *N* 倍的加速，这种情况叫作**超线性加速**（super-linear speedup）。为了验证这一点，用下面的作业描述和一个小缓存创建三个作业：

   ```
   -L a:100:100,b:100:100,c:100:100
   ```

   分别在有 1、2、3 个 CPU 的系统上运行（`-n 1`、`-n 2`、`-n 3`）。然后再做一遍，但把每个 CPU 的缓存大小改为 100。随着 CPU 数量增加，你注意到性能有什么变化？用 `-c` 验证你的猜测，再用其他跟踪选项深入研究。

8. 模拟器另一个值得研究的方面是按 CPU 调度的选项，即 `-p` 选项。再次用两个 CPU 运行，并使用这样的三作业配置（`-L a:100:100,b:100:50,c:100:50`）。和上面你手动设置的亲和性（affinity）限制相比，这个选项表现如何？把“窥视间隔”（`-P`）改小或改大，性能会怎样变化？随着 CPU 数量增加，这种按 CPU 调度的方法效果如何？

9. 最后，你可以随意生成随机的工作负载（workload），看看能否预测出它们在不同处理器数量、缓存大小和调度选项下的性能。这样做下去，你很快就会成为**多处理器调度大师**，这可是件相当了不起的事。祝你好运！